MagDyn HRMS

Complete Salary Calculation

How a monthly salary slip is computed, end to end -
every input, formula, rate base, deduction and guard.

Engine includes/PayrollCalculator.php
Extras includes/payroll_extras.php
Day rules includes/helpers.php (attendance_classify)
Constants config/app.php
Entry points modules/payroll/calculate.php (preview), generate_slip.php (persist)

All amounts in this document are Indian Rupees, written as **Rs.**

1. The calculation pipeline

A slip is produced by one call to **PayrollCalculator::computePayroll()**, followed by **payroll_apply_extras()**. The order below is the order the code actually executes in - each step consumes the output of the ones above it.

#	Step	What it produces
0	Resolve effective salary	Fixed monthly CTC in force at month-end (point-in-time)
1	Build the month calendar	Calendar days, working days, working-day dates
2	Read + classify attendance	Full / half / short / absent days, late minutes, OT minutes
3	Credit paid leave	Approved paid leave + classified paid leave + admin paid leave
4	Split CTC into allowances	Basic, HRA, Conveyance, ... , Special Allowance residual
5	Prorate to days earned	Every allowance x earn ratio (paid days / calendar days)
6	Add overtime	OT hours x Basic hourly rate x 2
7	Gross earnings	Sum of all allowance lines
8	Statutory deductions	PF 12% of earned Basic (cap Rs. 1,800), ESI 0.75%
9	Discipline deductions	Late-arrival half day (Basic rate), late penalty (2x)
10	Extras	Benefits + bonuses (earn), cashless benefit + loan EMI (deduct)
11	Totals	Gross, total deductions, Net Pay = Gross - Deductions (floored at 0)

NOTE Steps 5 and 8 are linked: because allowances are prorated *before* PF is computed, PF is charged on **earned** Basic, not full-month Basic. Overtime and the late penalty deliberately use **full-month** Basic instead.

2. Configuration constants and settings

2.1 Statutory constants - `config/app.php`

Constant	Value	Meaning
PAYROLL_PF_EMPLOYEE	0.12	Employee PF = 12% of Basic
PAYROLL_PF_EMPLOYER	0.12	Employer PF match (mirrored from the employee figure)
PF cap basic (class const)	15,000	PF is capped at 12% of 15,000 = Rs. 1,800
PAYROLL_ESI_EMPLOYEE	0.0075	Employee ESI = 0.75% of monthly CTC
PAYROLL_ESI_EMPLOYER	0.0075	Employer ESI (set equal to the employee rate in this build)
PAYROLL_ESI_WAGE_LIMIT	21,000	ESI applies only while CTC is strictly below this
PAYROLL_WORKING_DAYS	26	Legacy flat figure - used only for a badge, not the maths
BASIC_FALLBACK_RATIO	0.40	Basic = 40% of CTC when no 'Basic' component exists

2.2 Timing settings (per shift, with global fallbacks)

Every timing threshold resolves in this order: the shift **stamped on the attendance row** at mark time, then the employee's current shift, then the legacy global setting. That is what lets a past month keep being calculated against the shift the employee actually worked, even after they move shifts.

Setting	Default	Used for
office_start_time	09:00	Late minutes are measured from here
office_end_time	18:00	Left-early test for the short-hours rule
daily_grace_minutes	15	Check-in after start + grace = Late

Setting	Default	Used for
monthly_grace_minutes	90	Late penalty triggers only past this monthly total
half_day_cutoff	start + 2h (11:00)	Check-in at/after this = Half Day, no late
ot_trigger_time	20:30	Check-out must reach this before any OT is credited
ot_baseline_time	18:15	OT minutes are counted from here, not from the trigger
lunch_start / lunch_end	13:00 - 13:30	Per-employee lunch batch; deducted from worked time
tea1 / tea2	11:00-11:15, 16:00-16:15	Two global tea breaks; deducted from worked time

RULE A shift with **ot_enabled = 0** (a straight shift) earns zero overtime. This is absolute: even an admin's manual OT-hours override is forced back to 0, so payroll always agrees with what the attendance sheet recorded.

3. Step 0 - Which salary applies to this month

A slip for a past month must use the salary that was in force **then**, never today's post-increment salary. `getSalaryForMonth()` resolves it point-in-time against the last day of the payroll month:

Point-in-time salary resolution

```
lastDay = last calendar day of the payroll month

1. sRow = latest salary_structures row  with effective_from  <= lastDay
   iRow = latest employee_increments row with effective_date <= lastDay
   -  both present  : the LATER event wins; on a tie the increment wins
   -  one present   : that one

2. neither present (month predates every salary event):
   salary = earliest increment's previous_salary   (original pre-increment pay)

3. still nothing: salary = employees.fixed_salary, else 0
```

NOTE The `is_current` flag on salary_structures is deliberately **not** used here. It marks today's active salary, which is the wrong answer for a historical month.

If the resolved fixed salary is 0 or less, no slip can be generated - the employee is reported as *No salary structure* on the calculation register.

4. Step 1 - Working days for the month

Working days are computed from the real calendar, not from a flat constant, so the figure genuinely differs between February and August.

Working-day rule

```
for each day d in the month:
    if d is Sunday                      -> skip   (weekly off)
    if d is Saturday:
        satCount = satCount + 1
        if satCount is 1 or 3           -> skip   (1st & 3rd Saturday off)
        else                            -> WORKING (2nd, 4th, 5th Saturday)
    if d is a declared holiday          -> skip
    otherwise                           -> WORKING
```

- **Calendar days** (28-31) is the denominator for every per-day rate and for the earned-salary proration - *not* working days.
- **Working days** is the denominator for the absence test only: it decides how many days the employee was expected to be present.
- The Saturday test runs *before* the holiday test, so a holiday falling on an already-off 1st/3rd Saturday is never double-counted.

CAVEAT The blue badge on the Salary Calculation screen shows `PAYROLL_WORKING_DAYS (26) - holidays`, which is the legacy flat figure. The engine itself uses the real calendar rule above, so the badge and the per-employee maths can disagree by a day or two. The badge is display only - it never enters a calculation.

5. Step 2 - How each worked day is classified

Every punched day (status On Time / Late / Half Day, with both a check-in and a check-out) is re-classified from the **actual times** - the stored status label is not trusted. First the net worked time is derived:

Net worked minutes

```
presence   = check_out - check_in                      (in minutes)
breaks     = the parts of lunch + tea break 1 + tea break 2
             that fall INSIDE [check_in, check_out]
net worked = presence - breaks

A full day = 480 net minutes (8 hours).   A half day = 240 minutes (4 hours).
```

Only the *overlap* of a break window with the presence window is subtracted, so an employee who leaves before the 16:00 tea break is never charged for it. A shift configured with no breaks and no lunch subtracts nothing at all.

5.1 The classification ladder (first match wins)

#	Condition	Status	Deducted (ded_days)	Late applies?
1	net < 4h	short	(8h - net) / 8h	No
2	check-in at/after the half-day cutoff (~11:00), net >= 4h	half reason: late_arrival	max(0.5, (8h - net)/8h)	No
3	net exactly 4h	half reason: half_worked	(8h - net) / 8h = 0.5	No
4	net >= 8h, **or** stayed until/after office end	full	0	Yes
5	4h < net < 8h and left before office end	present	(8h - net) / 8h	Yes

- **Why rule 4 charges nothing:** if the employee stayed to the end of the shift, any shortfall against 8h is caused by a late check-in - which the late penalty already charges. Deducting short hours as well would punish the same lateness twice.
- **Why rules 1-3 set late = false:** the day has already lost half a day or more, so stacking the late penalty on top would be a second charge for the same tardiness.
- A day is counted as **late** only when check-in > shift start + daily grace, and the minutes banked into the monthly pool are the full *(check-in - shift start)*, grace included.

5.2 Aggregates handed to payroll

Field	Meaning
present	full + present + half + OD + Comp Off - every day that is 'not absent'
half_ded_days	Total half-day equivalent to charge (sum of ded_days for 'half' days)
late_half_ded_days	The part of the above caused by a late arrival (rule 2) - priced off Basic
short_ded_days	Total shortfall in days for 'short' and 'present' days
late_minutes	Monthly late pool (only from days where late = true)
ot_hours	Sum of OT minutes / 60 across On Time, Late, Half Day, OD
leave / paid_leave	Attendance rows with status Holiday - counted as paid
absent	Attendance rows with status Absent

5.3 Overtime minutes

OT minute rule

```
OT is credited for a day only when BOTH hold:
   the shift has ot_enabled = 1 AND has both an OT trigger and an OT baseline
   check_out >= ot_trigger_time            (e.g. 20:30)

OT minutes for the day = check_out - ot_baseline_time     (e.g. from 18:15)
```

So the trigger is a **gate** and the baseline is the **meter**: an employee who leaves at 20:29 earns nothing, while one who leaves at 20:31 is paid from 18:15 - 2 hours 16 minutes.

6. Step 3 - Paid leave, and what counts as absent

Three independent sources can convert a day into **paid leave**, so it is excluded from Loss of Pay:

Source	Condition	Applied
Approved leave request	leave_requests.status = approved **and** leave_types.is_paid = 1, intersected with the month's working days	Before the absence count
Classified absence	An Absent attendance row marked leave_classification = 'paid' on the Attendance Report, restricted to working days	Before the absence count
Admin entry on the form	The *Paid leaves* field on Generate Slip - converts up to that many **absent** days into paid leave	After the absence count

Unpaid leave types (is_paid = 0, e.g. Loss of Pay) are deliberately excluded from the first source and therefore remain LOP. Each working day is credited once even if several approved requests overlap it.

Absence

```
absentDays = working days - present days - paid leave days      (floored at 0)

manualPaid = min( admin paid-leave days, floor(absentDays) )

absentDays = absentDays - manualPaid

lopDays    = absentDays          <- the final Loss of Pay figure
```

Example: 4 absent days with 2 paid leaves entered by the admin leaves 2 LOP days.

7. Step 4 - Splitting the CTC into allowances

Rows in `salary_components` with type = allowance are applied to the monthly CTC in `sort_order`. A component is either a **percentage** of CTC or a **flat** amount. Components literally named 'PF' or 'ESI' are skipped here - those are deductions, computed authoritatively later.

Seeded component	Type	Value	On Rs. 30,000 CTC
Basic	percentage	55.00 %	16,500.00
HRA	percentage	25.00 %	7,500.00
Conveyance allowance	percentage	5.00 %	1,500.00
Vehicle allowance	percentage	5.00 %	1,500.00
Product Incentive	percentage	10.00 %	3,000.00
Total		**100.00 %**	**30,000.00**

7.1 Basic resolution and the residual

Basic + residual

```
Basic = the allowance whose NAME CONTAINS 'basic'
        (exact match on 'Basic Salary' preferred; else the first fuzzy match)

if NO such component exists:
        Basic = 40% of CTC, inserted as a new 'Basic Salary' line

Special Allowance = CTC - sum(all allowance lines)      (added only when > 0)
Variable Pay      = added as its own line when > 0
```

IMPORTANT Basic is the single most load-bearing number on the slip: it drives PF, the overtime rate and both late-related deductions. If the Basic component is ever deleted, the engine silently falls back to 40% of CTC and all three of those figures change.

8. Step 5 - Earned-salary proration (the core idea)

This build pays for **days earned** rather than paying the full CTC and then subtracting an 'Absent Deduction'. Both models arrive at the same take-home, but this one states it honestly: every line on the slip shows what was actually earned.

Proration

```
unpaidDays = lopDays
           + (half_ded_days - late_half_ded_days)      <- genuine half days only
           + short_ded_days

paidDays   = calendarDays - unpaidDays
earnRatio  = paidDays / calendarDays

if earnRatio < 1:
     every allowance line = round(line x earnRatio, 2)
     Basic                = round(Basic x earnRatio, 2)   -> PF follows earned wages
```

- Weekly offs and holidays are **not** subtracted. An employee who works every working day is paid 31/31 and receives the full CTC.
- Because absence is now priced into the earnings, the Absent / Half Day / Short Hours **deduction lines no longer appear on the slip**. Keeping them would charge the same absence twice. The amounts are still computed and stored in the attendance summary so the reports can show what each shortfall was worth.
- Half days caused by a **late arrival** are excluded from unpaidDays on purpose - they are charged separately against Basic (section 10.1) rather than being prorated out of every component at the CTC rate.

9. The two rate bases

This is the detail most often misread. The system prices money *out* and money *in* from two different wage bases.

Rate	Formula	Charged / paid on
perDay (CTC base)	CTC / calendarDays	Absent, half-day and short-hours amounts - i.e. everything folded into the proration
perHour	perDay / 8	Reporting figure on the attendance summary
basicPerDay (Basic base)	$\text{Basic}_{\text{full month}}$ / calendarDays	The late-arrival half day
basicPerHour	basicPerDay / 8	Overtime pay and the monthly late penalty

SYMMETRY $\text{Basic}_{\text{full month}}$ is captured *before* proration. Overtime and the late penalty are therefore priced off the standard, unprorated wage rate - a late hour is charged against exactly the same base that an overtime hour is paid from.

10. Step 6-9 - Overtime and every deduction

10.1 Overtime pay

Overtime

```
otHours  = manual override if the admin entered one, else the attendance total
           (forced to 0 when the shift has OT switched off)

otAmount = otHours x basicPerHour x 2

Slip line:  "Overtime (6.5 hrs)"          <- an EARNING
```

10.2 Provident Fund

PF

```
only when employees.pf_enabled = 1

pfEmployee = min( earnedBasic x 12% ,  15,000 x 12% )
           = min( earnedBasic x 12% ,  1,800.00 )
pfEmployer = pfEmployee

Slip line:  "Provident Fund (PF)"         <- a DEDUCTION
```

The employer share is an employer cost - it is reported separately and is never taken out of the employee's net pay.

10.3 ESI

ESI

```
only when employees.pf_enabled = 1  AND  CTC < 21,000

esiEmployee = CTC x 0.75%
esiEmployer = CTC x 0.75%

Slip line:  "ESI (Employee)"              <- a DEDUCTION
```

NOTE ESI is gated by the *same* pf_enabled toggle as PF. Switching PF off for an employee therefore stops both statutory deductions. Note also that ESI is charged on the full CTC, not on the prorated earned amount.

CAVEAT The badge on the Salary Calculation screen reads *'ESI: 0.75% + 3.25%'*, but `PAYROLL_ESI_EMPLOYER` is set to 0.0075 in config/app.php - so this build actually applies 0.75% + 0.75%. The badge text is stale; the constant is what the engine uses.

10.4 Half day caused by a late arrival

Late-arrival half day

```
lateHalfDeduction = late_half_ded_days x basicPerDay

Slip line:  "Half Day - Late Arrival (1 day)"        <- a DEDUCTION
```

This is the only place such a day is charged. It was held out of the proration in step 5 precisely so it could be priced against Basic here instead of against the full CTC.

10.5 Monthly late penalty

Late penalty

```
if totalLateMinutes > monthlyGrace   (default 90 minutes):

     deductableMinutes = totalLateMinutes x 2      <- the FULL total, doubled
     lateDeduction     = deductableMinutes x (basicPerHour / 60)

Slip line:  "Late Deduction (100 min, 2x rate)"      <- a DEDUCTION
```

CLIFF EDGE Once the grace is exceeded the **entire** monthly late total is charged at double rate - not merely the minutes beyond the grace. 91 late minutes is charged as 182 minutes. Below the grace, nothing at all is charged.

10.6 Component deductions

Any `salary_components` row with type = deduction is applied as a percentage of CTC or a flat amount - except rows literally named 'PF' or 'ESI', which are skipped so the authoritative calculations in 10.2 and 10.3 win.

11. Step 10 - Benefits, bonuses and loans

`payroll_apply_extras()` appends extra line items and re-sums the two totals. It changes **no** core formula - gross is still the sum of allowances and net is still gross minus deductions.

11.1 Benefit funds - an EARNING

Rule	Behaviour
Eligibility	status = active, and the benefit is live in this payroll month
End-date guard	Once end_date has passed, the benefit never appears again - in both recurring and legacy modes
Recurring window	start_date must be on or before month-end; frequency then decides: quarterly / half-yearly match the period, annual matches the month, monthly always matches
Legacy mode	No start_date: matches only when effective_month is exactly this month
Occurrences	weekly = active days / 7, fortnightly = active days / 14 (min 1); everything else pays once
Amount	amount x occurrences, labelled `[BENEFIT]`

NOTE Cashless benefits net to zero. A benefit with payment_mode = 'cashless' is paid directly to the provider (insurer, education fund), not handed to the employee. It is therefore added as an earning *and* subtracted as a matching deduction '<name> (Cashless - paid to provider)'. Take-home is unchanged; the slip shows the full value earned. A **cash** benefit has no matching deduction and does raise take-home.

11.2 Bonuses and incentives - an EARNING

Rows in `employee_bonuses` with status = approved and payroll_month / payroll_year matching this run, labelled `[BONUS]`. Types: Monthly Bonus, Performance Incentive, Festival Bonus, Overtime Incentive, One-time Reward.

11.3 Loans and advances - a DEDUCTION

Loan EMI

```
for each active loan with monthly_deduction > 0:

  skip if date_given is after this month's end       (not yet disbursed)

  totalDue       = principal + interest
  returnedBefore = sum of what PRIOR generated slips actually deducted
  remaining      = totalDue - returnedBefore
  skip if remaining <= 0                             (already cleared)

  if this is the final instalment OR remaining <= EMI:
        deduct = remaining          <- clears the exact balance
  else: deduct = min(EMI, remaining)
```

Settling the final instalment against the exact remaining balance is what stops rounding drift from leaving a stray few rupees pending forever.

12. Step 11 - Final totals

Totals

```
grossEarnings    = sum of every allowance line
                   (components + special allowance + variable pay + overtime
                    + benefits + bonuses)

totalDeductions = sum of every deduction line
                   (component deductions + PF + ESI + late-arrival half day
                    + late penalty + cashless benefits + loan EMIs)

netPay           = max( 0 , grossEarnings - totalDeductions )

Employer cost    = grossEarnings + pfEmployer + esiEmployer
```

NOTE Net pay is floored at zero. If deductions were ever to exceed gross - a large loan EMI in a heavy-LOP month - the slip shows 0 rather than a negative figure, and the shortfall is not automatically carried forward.

13. Worked example, end to end

Scenario - August 2026 (31 calendar days). Monthly CTC Rs. 30,000. PF enabled. Standard shift 09:00-18:00, OT enabled, seeded salary components.

13.1 Calendar

Item	Value	Working
Calendar days	31	August
Sundays	5	2, 9, 16, 23, 30 - all off
Saturdays off	2	1st (Aug 1) and 3rd (Aug 15); the 8th, 22nd, 29th are working
Declared holidays	1	Aug 15 - already an off Saturday, so no further reduction
Working days	**24**	31 - 5 - 2

13.2 Attendance for the month

Days	What happened	Classified	ded_days
20	Worked a full shift	full	0
1	Net 6h, left before 18:00	present	(8-6)/8 = 0.25
1	Checked in 11:30, worked 5h	half (late_arrival)	max(0.5, 0.375) = 0.50
1	Approved paid Casual Leave	paid leave	0
1	Absent, unexplained	absent	1.00 (LOP)
24	**Total working days**		

Also recorded: late arrivals on 3 of the full days totalling **100 minutes**, and **6.5 hours** of overtime.

13.3 Days earned

Proration for the example

```
present    = 20 full + 1 present + 1 half = 22
paid leave = 1        (approved casual leave)
absentDays = 24 - 22 - 1 = 1        -> lopDays = 1

unpaidDays = 1.00 (LOP)
           + (0.50 half - 0.50 late-arrival half)   = 0.00
           + 0.25 (short)
           = 1.25

paidDays   = 31 - 1.25 = 29.75
earnRatio  = 29.75 / 31 = 0.959677
```

13.4 Rates

Rate	Working	Value (Rs.)
Basic (full month)	30,000 x 55%	16,500.0000
perDay (CTC base)	30,000 / 31	967.7419
perHour	967.7419 / 8	120.9677
basicPerDay	16,500 / 31	532.2581
basicPerHour	532.2581 / 8	66.5323

13.5 Earnings

Line	Full month	x 0.959677	Earned (Rs.)
Basic (55%)	16,500.00		15,834.68
HRA (25%)	7,500.00		7,197.58
Conveyance allowance (5%)	1,500.00		1,439.52
Vehicle allowance (5%)	1,500.00		1,439.52
Product Incentive (10%)	3,000.00		2,879.03
Special Allowance	*0.00*		*not added - components total 100%*
Prorated CTC	**30,000.00**		**28,790.33**
Overtime (6.5 hrs)	6.5 x 66.5323 x 2	not prorated	864.92
[BENEFIT] Health Insurance (cashless)	1,000.00	not prorated	1,000.00
[BONUS] Performance Incentive	2,000.00	not prorated	2,000.00
GROSS EARNINGS			**32,655.25**

13.6 Deductions

Line	Working	Amount (Rs.)
Provident Fund (PF)	min(15,834.68 x 12% = 1,900.16 , cap 1,800.00)	1,800.00
ESI (Employee)	not applicable - CTC 30,000 is not below 21,000	0.00
Half Day - Late Arrival (1 day)	0.50 x 532.2581	266.13
Late Deduction (100 min, 2x rate)	100 x 2 = 200 min x (66.5323 / 60)	221.77

Line	Working	Amount (Rs.)
Health Insurance (Cashless - paid to provider)	mirrors the benefit earning	1,000.00
Advance Deduction #1	monthly EMI on an active advance	2,500.00
TOTAL DEDUCTIONS		**5,787.90**

13.7 Net pay

Gross earnings	**32,655.25**
Less: total deductions	**- 5,787.90**
NET PAY	**Rs. 26,867.35**

Employer cost: gross 32,655.25 + employer PF 1,800.00 + employer ESI 0.00 = **Rs. 34,455.25**.

Small rounding differences (a rupee or less) can appear because each allowance line is rounded to 2 decimals individually before being summed.

14. Variant - a salary where ESI applies and PF is not capped

Same month, CTC **Rs. 18,000**, perfect attendance (earnRatio = 1.0), no overtime, no extras, PF enabled.

Line	Working	Amount (Rs.)
Basic (55%)	18,000 x 55%	9,900.00
HRA (25%)	18,000 x 25%	4,500.00
Conveyance allowance (5%)	18,000 x 5%	900.00
Vehicle allowance (5%)	18,000 x 5%	900.00
Product Incentive (10%)	18,000 x 10%	1,800.00
Gross earnings		**18,000.00**
Provident Fund (PF)	min(9,900 x 12% = 1,188.00 , cap 1,800.00) - **under the cap**	1,188.00
ESI (Employee)	18,000 < 21,000, so 18,000 x 0.75%	135.00
Total deductions		**1,323.00**
NET PAY		**16,677.00**

Employer side: PF 1,188.00 + ESI 135.00 = Rs. 1,323.00, giving an employer cost of Rs. 19,323.00. The moment this employee's CTC reaches Rs. 21,000, ESI stops entirely - there is no taper.

15. Guards, edge cases and stored data

15.1 Guards enforced before a slip can be generated

Guard	Behaviour
Future month	Blocked. Payroll runs only for the current or a past month - otherwise a loan EMI would be deducted for a month that has not happened. The Generate buttons are disabled on the register too.
Before joining date	The employee is listed but skipped - no calculation, no slip.
No salary structure	Listed as 'No Salary' and skipped.
Inactive employee	Only status = Active employees are processed.

Guard	Behaviour
Duplicate slip	One slip per employee per payroll_month. A second attempt prompts to regenerate, which updates the existing row in place.
Self-scoped user	An employee-scoped login sees only their own row.
CSRF	Generation is a POST with CSRF verification.

15.2 What is frozen onto the slip

Generation persists the computed figures so a slip never changes retroactively when master data is edited afterwards:

- **allowances** and **deductions_json** - the full line-item breakdown, in the component sort order that applied at generation time.
- **attendance_summary** (JSON) - working days, present, half, paid leave, absent, late minutes and grace, deductable late minutes, OT hours and rate, paid/unpaid days, earn ratio, full-month Basic, per-day and per-hour rates, calendar days, CTC.
- **shift_name** - read from the shift stamped on each attendance row, so a slip regenerated after the employee changes shift still names the shift they actually worked.
- Flat columns for reporting: basic, hra, conveyance, medical, special_allow, gross_earnings, pf_employee, pf_employer, esi_employee, esi_employer, total_deductions, net_pay, working_days, present_days, lop_days.
- Generation also writes back per-day **worked_hours** and **deduction_amount** onto the attendance rows themselves, as an audit trail.

15.3 Tables involved

Table	Role
employees	Status, joining date, pf_enabled toggle, shift, lunch batch, fallback fixed_salary
salary_structures	CTC history (effective_from); is_current marks today's salary only
employee_increments	Increment audit trail - new_salary, previous_salary, effective_date
salary_components	The earnings/deductions breakup and its sort order
attendance	Per-day status, in/out times, stamped shift_id, leave_classification
shifts	Start/end, grace, half-day cutoff, OT enable + trigger + baseline, breaks
holidays	Declared holidays excluded from working days
leave_requests + leave_types	Approved paid leave (is_paid = 1)
employee_benefits	Benefit funds - frequency, dates, cash / cashless
employee_bonuses	Approved bonuses for a specific payroll month
employee_loans	Active loans and advances - EMI, tenure, interest
salary_slips	The generated slip and its frozen JSON breakdown

15.4 Summary of every formula on one page

Every formula

```
workingDays    = calendar days - Sundays - 1st & 3rd Saturdays - holidays
netWorked      = (out - in) - overlapping break windows
unpaidDays     = LOP + genuine half days + short-hour shortfall
earnRatio      = (calendarDays - unpaidDays) / calendarDays

allowance_i    = (CTC x pct_i  or  flat_i) x earnRatio
specialAllow   = CTC - sum(allowances)                  (if > 0)
basicFullMonth = the 'basic' component at 100%, else CTC x 40%

perDay         = CTC / calendarDays
basicPerDay    = basicFullMonth / calendarDays
basicPerHour   = basicPerDay / 8

otAmount       = otHours x basicPerHour x 2
pfEmployee     = min(earnedBasic x 12%, 1800)           (pf_enabled only)
esiEmployee    = CTC x 0.75%                            (pf_enabled, CTC < 21000)
lateHalfDed    = lateHalfDays x basicPerDay
lateDed        = (totalLateMins x 2) x basicPerHour / 60  (only past 90 min)

GROSS          = sum(allowances) + OT + benefits + bonuses
DEDUCTIONS     = components + PF + ESI + lateHalfDed + lateDed
                 + cashless benefits + loan EMIs
NET PAY        = max(0, GROSS - DEDUCTIONS)
```